

Cluster Runbook - every mode, every test

How to run the PyTorch-Spark inference platform locally in each mode - tensor models and the NER pipeline, service and cluster execution, models from HDFS, from the master's file system or from the repository - and the complete test suite with the outputs of the last run.

Last simulation run: 12 tests, 12 PASS, 0 FAIL (deploy/airgap_sim_tests.ps1, logs under results/airgap_sim_*).
Companion document: NER_PIPELINE_END_TO_END_PROCESS.pdf.

Contents

1. Modes at a glance
2. One-time preparation
3. The air-gapped simulation environment
4. Run everything (automated)
5. Run each mode by hand
6. Test catalogue with results
7. Tensor-model modes and their tests
8. Moving to the real air-gapped cluster
9. Troubleshooting
10. Teardown and restoring the laptop

1. Modes at a glance

#	mode	what runs	model source	how to start
A	local single container (tensor models)	driver + executor in one container, Spark local[2]	repository / torchvision cache	<code>docker run ... python submit_job.py --master local[2]</code>
B	laptop Spark cluster (tensor models)	master + GPU worker (+ CPU workers) containers	repository	<code>docker compose -f deploy/docker-compose.laptop.yml up -d</code>
C	NER service mode (waiter/kitchen)	lean Spark master + worker, GPU model server	HDFS / master FS / repository	<code>airgap_sim profiles hdfs + service</code>
D	NER cluster mode (in-process)	Spark master + GPU worker carrying all NER deps	HDFS / master FS / repository	<code>airgap_sim profiles hdfs + cluster</code>
E	models from HDFS	WebHDFS fetch into node caches	<code>hdfs://hdfs-namenode:8020/models/weights</code>	<code>MODEL_STORE_URI=hdfs://...</code>
F	models from the master's file system	read-only mount on every node	<code>file:///mnt/models</code>	<code>MODEL_STORE_URI=file:///mnt/models + MODEL_FS_DIR</code>
G	models from the repository (legacy)	models/weights next to the code	models/weights	<code>MODEL_STORE_URI</code> unset
H	AWS single node / 2 nodes	g4dn (+ m5) with SSM	S3 upload of the repo	<code>deploy/run_aws_campaign.ps1</code> , <code>deploy/run_aws_modes.ps1</code>

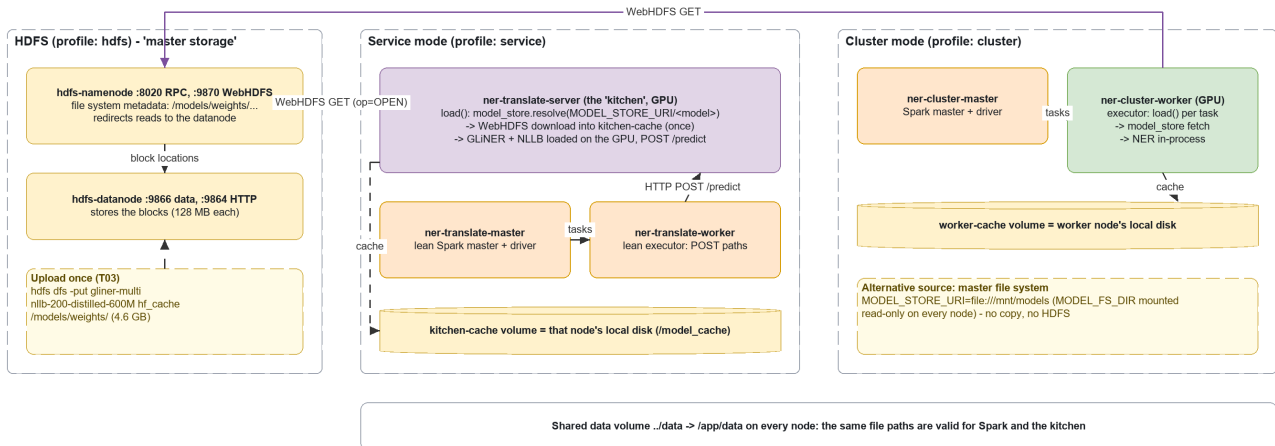
2. One-time preparation (this laptop)

item	state / command
Docker memory	WSL2 VM raised to 6 GB: <code>%USERPROFILE%\wslconfig [wsl2] memory=6GB</code> , then <code>wsl --shutdown</code> and restart Docker Desktop (original saved as <code>.wslconfig.bak-20260927</code>). The NER model server needs ~4 GB while loading.
Cassandra	stop during NER tests: <code>docker stop cassandra-web-ui cassandra-database</code> (start again afterwards)
images	<code>apache/hadoop:3.4.1</code> , <code>ner-translate-server:latest</code> , <code>spark-lean:latest</code> , <code>ner-translate-worker:latest</code> , <code>multi-model-inference:latest</code> (all present)
model weights (staging)	<code>D:\pytorch-spark-inference-platform_20260921\models\weights</code> (gliner-multi 2.2 GB, nllb-200-distilled-600M 2.4 GB, hf_cache 4 MB) - passed to the tests with <code>-WeightsSrc</code>
disk	the simulation adds ~10 GB inside Docker's disk (HDFS copy, node caches); removed with <code>-Wipe</code>

3. The air-gapped simulation environment

21. Air-gapped cluster simulation: models in HDFS, nodes fetch into a local cache

deploy/docker-compose.airgap_sim.yml - profiles hdfs / service / cluster; MODEL_STORE_URI selects where models come from



deploy/docker-compose.airgap_sim.yml

profile	services (container)	host ports
hdfs	hdfs-namenode (sim-hdfs-namenode), hdfs-datanode (sim-hdfs-datanode)	9870 WebHDFS / namenode UI
service	ner-translate-server (sim-ner-translate-server, GPU), ner-translate-master (sim-spark-master), ner-translate-worker (sim-spark-worker)	8001 kitchen API, 8083 Spark master UI, 4043 job UI
cluster	ner-cluster-master (sim-ner-cluster-master), ner-cluster-worker (sim-ner-cluster-worker, GPU)	8084 Spark master UI, 4044 job UI

variable	meaning	example
MODEL_STORE_URI	where every node loads models from	hdfs://hdfs-namenode:8020/models/weights file:///mnt/models (empty)
MODEL_FS_DIR	host folder mounted read-only at /mnt/models (the master's shared file system)	D:\...\models\weights
WEIGHTS_SRC	host folder uploaded into HDFS (mounted at /staging on the namenode)	D:\...\models\weights
MODEL_CACHE_DIR	node-local cache (set to /model_cache, a per-node volume)	/model_cache
MODEL_STORE_WEBHDFS / MODEL_STORE_USER	WebHDFS URL override / HDFS user (optional)	http://namenode:9870 / root

4. Run everything (automated)

```

.\deploy\airgap_sim_tests.ps1 # T01..T12
.\deploy\airgap_sim_tests.ps1 -Tests T05,T06,T07 # a subset
.\deploy\airgap_sim_tests.ps1 -WeightsSrc D:\models\weights # another staging folder
.\deploy\airgap_sim_tests.ps1 -Tests T12 -Wipe # teardown incl. HDFS data and caches

```

Each test prints PASS/FAIL and its evidence; full output goes to `results\airgap_sim_<timestamp>\<test>.log` with a `summary.txt/json`.

5. Run each mode by hand (PowerShell, repository root)

5.1 HDFS up and models uploaded (modes C/D with E)

```
$env:WEIGHTS_SRC = "D:\pytorch-spark-inference-platform_20260921\models\weights"
docker compose -f deploy/docker-compose.airgap_sim.yml --profile hdfs up -d
docker exec sim-hdfs-namenode hdfs dfsadmin -report # Live datanodes (1)
docker exec sim-hdfs-namenode hdfs dfs -mkdir -p /models/weights
docker exec sim-hdfs-namenode hdfs dfs -put -f /staging/gliner-multi /staging/nllb-200-distilled-600M /staging/hf_cache /models/weights/
docker exec sim-hdfs-namenode hdfs dfs -du -s -h /models/weights/*
start http://localhost:9870 # namenode UI -> Utilities -> Browse
```

5.2 Service mode with models from HDFS (C + E)

```
$env:MODEL_STORE_URI = "hdfs://hdfs-namenode:8020/models/weights"
docker compose -f deploy/docker-compose.airgap_sim.yml --profile hdfs --profile service up -d ner-translate-server
docker logs -f sim-ner-translate-server # [model_store] fetched ... -> Models loaded in ... s (Ctrl+C)
Invoke-RestMethod http://localhost:8001/health
docker compose -f deploy/docker-compose.airgap_sim.yml --profile hdfs --profile service up -d
docker exec sim-spark-master python submit_pipeline_job.py --pipeline ner_translate --input data/ner_samples `
--partitions 2 --execution-mode service --master spark://ner-translate-master:7077 --driver-memory 1g --executor-memory 512m
```

5.3 Call the model server directly (no Spark)

```
Invoke-RestMethod -Method Post -Uri http://localhost:8001/predict -ContentType application/json `
-Body '{"paths":["data/ner_samples/sample_text1.txt","data/ner_samples/sample_scan6.png"]}'
```

5.4 Master file-system mode (C + F)

```
$env:MODEL_FS_DIR = "D:\pytorch-spark-inference-platform_20260921\models\weights"
$env:MODEL_STORE_URI = "file:///mnt/models"
docker compose -f deploy/docker-compose.airgap_sim.yml --profile service up -d --force-recreate ner-translate-server
docker logs sim-ner-translate-server | Select-String 'model source' # model store file:///mnt/models
```

5.5 Cluster (in-process) mode with models from HDFS (D + E)

```
docker compose -f deploy/docker-compose.airgap_sim.yml --profile service stop # free memory for the executor
$env:MODEL_STORE_URI = "hdfs://hdfs-namenode:8020/models/weights"
docker compose -f deploy/docker-compose.airgap_sim.yml --profile hdfs --profile cluster up -d
docker exec sim-ner-cluster-master python submit_pipeline_job.py --pipeline ner_translate --input data/ner_samples `
--partitions 1 --execution-mode cluster --master spark://ner-cluster-master:7077 --driver-memory 1g --executor-memory 4g
docker exec sim-ner-cluster-worker bash -c "grep -rh model_store /opt/spark/work | tail" # executor fetched from HDFS
```

5.6 Repository mode (G) - original behaviour

Leave MODEL_STORE_URI empty and put the three folders in models/weights of the repository (robocopy <src> models\weights /E, add models/weights/ to .git/info/exclude). Start the service or cluster profile as above.

6. Test catalogue with results

test	what it proves	pass criteria	result
T01	Prerequisites	Docker VM memory >= 5.5 GB, 4 images, weights present, CUDA visible inside containers	PASS (31.2 s)
T02	HDFS up	namenode healthy, 1 live datanode	PASS (47.9 s)
T03	Upload models to HDFS	hdfs dfs -put the three folders; HDFS byte count == local byte count (hf_cache: present - its snapshot symlinks are stored as files)	PASS (625.4 s)
T04	model_store over WebHDFS	fetches file sha256 == original; second call is a cache hit; missing path -> FileNotFoundError; unknown host -> ConnectionError	PASS (61.7 s)
T05	Kitchen with models from HDFS	logs 'model source: model store hdfs://', '[model_store] fetched', models loaded on GPU, /health ok	PASS (461.8 s)
T06	Direct /predict	a text file and a scanned PNG return language + entities	PASS (70.6 s)
T07	Spark job, service mode	6 documents processed through the lean cluster	PASS (96.6 s)
T08	Node cache reuse	restart -> 2 cache hits, no download	PASS (206.5 s)
T09	HDFS outage, warm cache	HDFS stopped; kitchen restarts from cache and answers /predict	PASS (304.3 s)
T10	Master file-system mode	file:///mnt/models, no fetch; Spark job succeeds	PASS (359.8 s)
T11	Cluster mode, executor fetches from HDFS	executor log shows model_store fetch; 6 documents processed in-process	PASS (620.9 s)
T12	Teardown	services stopped; volumes kept (or removed with -Wipe)	PASS (30 s)

T01 - Prerequisites: PASS (31.2 s)

```
Docker VM memory: 5.8 GB (>= 5.5 GB needed for the model server + HDFS + Spark)
image ok: apache/hadoop:3.4.1
image ok: ner-translate-server:latest
image ok: spark-lean:latest
image ok: ner-translate-worker:latest
weights ok: gliner-multi\gliner_config.json
weights ok: nllb-200-distilled-600M\config.json
weights ok: hf_cache\models--microsoft--mdeberta-v3-base
GPU in containers: True NVIDIA GeForce GTX 1650
```

T02 - HDFS up: PASS (47.9 s)

```
Live datanodes: 1; WebHDFS: http://localhost:9870
```

T03 - Upload models to HDFS: PASS (625.4 s)

```
gliner-multi: local 2311737808 bytes, HDFS 2311737808 bytes
nllb-200-distilled-600M: local 2482656260 bytes, HDFS 2482656260 bytes
hf_cache: local 4305697 bytes, HDFS 8611353 bytes
```

T04 - model_store over WebHDFS: PASS (61.7 s)

```
sha256 e25f61d91620df84aae8076811ee592e926e94d341b82e7bc1be359718f83017
[model_store] cache hit /tmp/t04/hdfs-namenode/models/weights/gliner-multi/gliner_config.json
expected error: FileNotFoundError model store: /models/weights/does-not-exist not found in HDFS at http://hdfs-namenode:9870 (HTTP 404)
expected error: ConnectionError model store: cannot reach WebHDFS at http://no-such-namenode:9870: [Errno -2] Name or service not known
sha256 of the local original: e25f61d91620df84aae8076811ee592e926e94d341b82e7bc1be359718f83017
```

T05 - Kitchen with models from HDFS: PASS (461.8 s)

```
INFO:ner_translate.serve:Models loaded in 416.9s
[ner_translate] model source: model store hdfs://hdfs-namenode:8020/models/weights
[model_store] fetched http://hdfs-namenode:9870/models/weights/gliner-multi -> /model_cache/hdfs-namenode/models/weights/gliner-multi 2311.7 MB in 118.3 s (20 MB/s)
[model_store] fetched http://hdfs-namenode:9870/models/weights/nllb-200-distilled-600M -> /model_cache/hdfs-namenode/models/weights/nllb-200-distilled-600M 2482.7 MB in 130.3 s (19 MB/s)
[model_store] fetched http://hdfs-namenode:9870/models/weights/hf_cache -> /model_cache/hdfs-namenode/models/weights/hf_cache 8.6 MB in 0.8 s (10 MB/s)
GPU detected: NVIDIA GeForce GTX 1650
```

T06 - Direct /predict: PASS (70.6 s)

```
sample_text1.txt: language=en translated=False entities=28
sample_scan6.png: language=en translated=False entities=8
```

T07 - Spark job, service mode: PASS (96.6 s)

```
[submit_pipeline_job] execution-mode=service -> HTTP model server at http://ner-translate-server:8000
Processed 6 document(s):
  sample_scan6.png: lang=en translated=False entities=8
  sample_text1.txt: lang=en translated=False entities=27
  sample_text2.txt: lang=en translated=False entities=33
  sample_text3.txt: lang=en translated=False entities=6
  sample_text4.txt: lang=hi translated=True entities=14
  sample_text5.txt: lang=mr translated=True entities=24
```

T08 - Node cache reuse: PASS (206.5 s)

```
[model_store] cache hit /model_cache/hdfs-namenode/models/weights/gliner-multi
[model_store] cache hit /model_cache/hdfs-namenode/models/weights/nllb-200-distilled-600M
[model_store] cache hit /model_cache/hdfs-namenode/models/weights/hf_cache
INFO:ner_translate.serve:Models loaded in 161.9s
```

T09 - HDFS outage, warm cache: PASS (304.3 s)

```
HDFS stopped; /predict -> language=en
HDFS restarted
```

T10 - Master file-system mode: PASS (359.8 s)

```
INFO:ner_translate.serve:Models loaded in 245.4s
[ner_translate] model source: model store file:///mnt/models
Processed 6 document(s):
  sample_scan6.png: lang=en translated=False entities=8
  sample_text1.txt: lang=en translated=False entities=27
  sample_text2.txt: lang=en translated=False entities=33
  sample_text3.txt: lang=en translated=False entities=6
  sample_text4.txt: lang=hi translated=True entities=14
  sample_text5.txt: lang=mr translated=True entities=24
```

T11 - Cluster mode, executor fetches from HDFS: PASS (620.9 s)

```
[submit_pipeline_job] execution-mode=cluster -> in-process (mod.load/mod.run) on spark://ner-cluster-master:7077
Processed 6 document(s):
executor: [ner_translate] model source: model store hdfs://hdfs-namenode:8020/models/weights
executor: [model_store] fetched http://hdfs-namenode:9870/models/weights/gliner-multi -> /model_cache/hdfs-namenode/models/weights/gliner-multi 2311.7 MB in 72.7 s (32 MB/s)
executor: [model_store] fetched http://hdfs-namenode:9870/models/weights/nllb-200-distilled-600M -> /model_cache/hdfs-namenode/models/weights/nllb-200-distilled-600M 2482.7 MB in 109.3 s (23 MB/s)
executor: [model_store] fetched http://hdfs-namenode:9870/models/weights/hf_cache -> /model_cache/hdfs-namenode/models/weights/hf_cache 8.6 MB in 0.9 s (10 MB/s)
```

T12 - Teardown: PASS (30 s)

```
stopped (volumes kept: hdfs-name, hdfs-data, kitchen-cache, worker-cache)
```

7. Tensor-model modes and their tests

The 10 built-in models and BYOM plugins use `submit_job.py` / the benchmark scripts. These were verified earlier on this laptop (Windows and WSL2 legs) and on AWS; commands and reference results:

```
# A - local single container (Linux image, GPU)
docker run --rm --gpus all -v "${PWD}:/work" -w /work multi-model-inference:latest `
python submit_job.py --model example_mlp --samples 20000 --mode gpu_only --master local[2] [--engine udf]

# B - laptop Spark cluster (master + GPU worker; --scale spark-cpu-worker=N adds CPU workers)
docker compose -f deploy/docker-compose.laptop.yml up -d
docker exec spark-master python benchmark/cluster_benchmark.py --device-mode hybrid --partitions 4 `
--signal-samples 3000 --image-samples 100 --detection-samples 30

# benchmark phases 1-10 (logs in results/campaign_<date>/<leg>)
docker exec -e LEG=ws12_docker spark-master bash benchmark/run_campaign.sh
python benchmark/summarize_campaign.py results/campaign_20260926

# execution-mode statistics (RDD / pandas UDF / predict_batch_udf / Triton), analysis and charts
python benchmark/spark_modes_stats.py --master spark://<master>:7077 --out results/modes_<date>/<run>
python benchmark/analyze_modes_stats.py results/modes_<date>/<run>
```

test	reference result	source
example_mlp RDD / pandas UDF (AWS T4)	3,964 / 1,774 samples/s	results/campaign_20260926/summary.md
10 models cpu_only / gpu_only / hybrid (AWS T4, 3k)	904 / 1,444 / 1,449 samples/s	same
same GPU, WSL2 vs Windows (parallel CUDA streams)	20,618 vs 3,797 samples/s	same
2-node: tasks per node, gpu_only fallback, GPU-aware	4/4 split; 6 of 8 partitions on CPU; 8/8 on GPU	SPARK_INFERENCE_MODES_AND_TRITON_ARCHITECTURE_20260926.docx
Triton vs in-process GPU inference stage (ResNet18)	3.1 s vs 7.0 s	same

8. Moving to the real air-gapped cluster

step	detail
images	docker save apache/hadoop:3.4.1 (3.3 GB), ner-translate-server (9.3 GB), spark-lean (2.2 GB), ner-translate-worker (28 GB, cluster mode only) -> transfer -> docker load on each node
models	hdfs dfs -put gliner-multi nllb-200-distilled-600M hf_cache /models/weights/ on the real cluster (or copy the three folders to the master's shared folder)
code	repository without weights/results; models/model_store.py + the updated pipeline.py and plugin_loader.py
node configuration	MODEL_STORE_URI=hdfs://<namenode>:8020/models/weights (or file:///<shared path>), MODEL_CACHE_DIR on a local disk with >= 5 GB free, MODEL_STORE_WEBHDFS if WebHDFS is not on <namenode>:9870
network	nodes -> namenode 9870 (WebHDFS) AND -> every datanode 9864 (reads are redirected there); Spark 7077/8080 + ephemeral ports inside the cluster; Spark nodes -> kitchen 8000
security	WebHDFS with simple auth (user.name) is what model_store supports; on a Kerberised HDFS use HttpFS/Knox with a service account or the master file-system mode
verification	run the same checks as T02-T11 on the real nodes (hdfs dfsadmin -report, hdfs dfs -du, kitchen /health, one submit_pipeline_job)

9. Troubleshooting (including what came up while building this simulation)

symptom	cause	fix
namenode exits: 'storage directory does not exist or is not accessible'	volume owned by root, daemon runs as user hadoop	user: root + dfs.namenode.name.dir on the volume (as in the compose file)
'docker compose up -d' blocks in a PowerShell helper	an advanced PowerShell function binds -d to -Debug	plain function with @args (airgap_sim_tests.ps1)
upload to HDFS is slow	Windows -> Docker file sharing (~10-20 MB/s)	expected on a laptop; real clusters upload from a Linux node
kitchen restarts while loading / container killed	Docker VM memory	6 GB WSL2 limit, Cassandra stopped, one heavy profile at a time
'cannot reach WebHDFS'	HDFS down or wrong host/port	start HDFS; check MODEL_STORE_URI / MODEL_STORE_WEBHDFS

symptom	cause	fix
executor cannot open http://hdfs-datanode:9864	datanode not reachable from the node	open port 9864 / fix datanode hostname (dfs.datanode.hostname)
service mode says 'no service_url'	manifest lacks service_url	keep ner-translate-server as the kitchen's hostname or edit the manifest
old code runs in the container	prebuilt images carry older code	the simulation mounts inference/, models/ and submit_pipeline_job.py from the repository
GLiNER: 'couldn't connect to huggingface.co ... mdeberta-v3-base'	GLiNER's backbone config/tokenizer not in the Hugging Face cache (offline)	put hf_cache into the model store; pipeline.load() installs it (fixed in this release - the old compose files set HF_HUB_CACHE instead)
'Repo id must be in the form ...' naming /app/models/weights/gliner-multi	MODEL_STORE_URI empty -> repository mode, but the repository has no weights	set MODEL_STORE_URI before docker compose up (the test script defaults it to the HDFS store)
'cannot stop container ... PID is zombie'	OCR / tokenizer child processes not reaped (Python server is PID 1)	init: true on every Python service (tini reaps them) - set in the compose file
cluster mode: every document 'No such file or directory: data/...'	relative input paths resolved in the executor's work folder	fixed: submit_pipeline_job.py sends absolute paths

10. Teardown and restoring the laptop

```

.\deploy\airgap_sim_tests.ps1 -Tests T12           # stop, keep HDFS data + caches
.\deploy\airgap_sim_tests.ps1 -Tests T12 -Wipe     # stop and delete HDFS data + caches (~10 GB)
docker start cassandra-database cassandra-web-ui
# optional: give Windows its memory back
Copy-Item $env:USERPROFILE\.wslconfig.bak-20260927 $env:USERPROFILE\.wslconfig; wsl --shutdown # then start Docker Desktop

```